

Support Vector Machines

The widest street: geometry, intuition, kernels and practice

Sayan CHAKI

Practical Machine Learning
LIRIS, Université Lyon 2

By the end of this session you should be able to:

1. Explain, with a picture, why an SVM chooses the **widest** separating street.
2. Compute a signed distance, a margin and a hinge loss by hand.
3. Say what a **support vector** is and why the others do not matter.
4. Explain the role of C (price of mistakes) and γ (reach of similarity).
5. Explain the **kernel trick** with a concrete numerical check.
6. Build, tune and evaluate an SVM in scikit-learn **without data leakage**.

Roadmap

1. The big idea: the widest street
2. Geometry of a linear classifier
3. Maximum margin: the hard-margin SVM
4. Support vectors and the dual view
5. Soft margin: allowing mistakes at a price
6. Kernels: straight lines in a curved world
7. Practice: preprocessing, evaluation and code
8. Comparative study and exercises
9. Wrap-up

Where we are

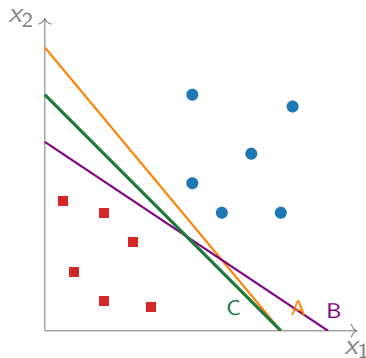
1. The big idea: the widest street
2. Geometry of a linear classifier
3. Maximum margin: the hard-margin SVM
4. Support vectors and the dual view
5. Soft margin: allowing mistakes at a price
6. Kernels: straight lines in a curved world
7. Practice: preprocessing, evaluation and code
8. Comparative study and exercises
9. Wrap-up

A story to start with

Imagine a map with **blue houses** and **red houses**. The city wants to build a straight road that keeps the two neighbourhoods apart.

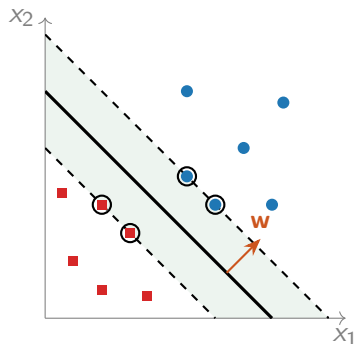
- Many roads are possible.
- Tomorrow, new houses will be built near the existing ones.
- Which road is least likely to end up with a new house on the wrong side?

Answer: the road that stays as far as possible from both neighbourhoods, i.e. the **widest empty street**.



All three roads A, B, C separate the data.
Which do you trust?

The SVM answer: maximise the width of the street



solid: $f(\mathbf{x}) = 0$ dashed: $f(\mathbf{x}) = \pm 1$

Vocabulary via the street picture

- **Centre line** = decision boundary, $f(\mathbf{x}) = 0$.
- **Gutters** (dashed) = margin boundaries, $f(\mathbf{x}) = \pm 1$.
- **Width of the street** = the margin.
- **Houses touching the gutters** (circled) = support vectors.
- **w** points across the street, towards the positive side.

Intuition

Only the circled houses hold the street in place.
Move any other house (without crossing a gutter):
the street stays put.

Why should a wide street generalise better?

- **Robustness to noise.** New points are old points plus a little noise. If the street has half-width 0.7 and measurement noise is at most 0.3, a perturbed training point can never jump across the centre line.
- **Fewer ways to cheat.** A narrow road can squeeze between points in many odd ways; a wide road has little room to wiggle. Less flexibility means less overfitting.
- **Only the difficult points matter.** The decision depends on the few borderline examples, not on the thousands of easy ones far away.

A classroom analogy

If you had to separate two groups of students standing in a courtyard with a rope, you would put the rope in the middle of the gap, not brushing against one group. That is exactly the maximum-margin idea.

Where we are

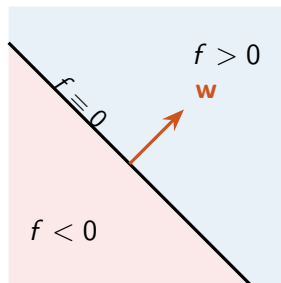
1. The big idea: the widest street
2. Geometry of a linear classifier
3. Maximum margin: the hard-margin SVM
4. Support vectors and the dual view
5. Soft margin: allowing mistakes at a price
6. Kernels: straight lines in a curved world
7. Practice: preprocessing, evaluation and code
8. Comparative study and exercises
9. Wrap-up

The linear score

We observe $\mathbf{x}_i \in \mathbb{R}^p$ with labels $y_i \in \{-1, +1\}$. A linear classifier computes

$$f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b, \quad \hat{y} = \text{sign}(f(\mathbf{x})).$$

- **w**: **direction** of the normal to the boundary (which way is “positive”).
- **b**: **offset**, it slides the boundary without rotating it.
- $f(\mathbf{x}) > 0$: predicted positive; $f(\mathbf{x}) < 0$: predicted negative.
- $|f(\mathbf{x})|$ grows as we move away from the boundary: a *confidence-like* score.



Why is $\mathbf{w}$ perpendicular to the boundary?

Take two points $\mathbf{x}_a, \mathbf{x}_b$ on the boundary: $\mathbf{w}^\top \mathbf{x}_a + b = 0$ and $\mathbf{w}^\top \mathbf{x}_b + b = 0$. Subtract:

$$\mathbf{w}^\top (\mathbf{x}_a - \mathbf{x}_b) = 0.$$

The vector $\mathbf{x}_a - \mathbf{x}_b$ lies along the boundary, and its dot product with $\mathbf{w}$ is zero. So $\mathbf{w}$ is orthogonal to every direction inside the boundary.

Intuition

Think of f as the height of a tilted plane over the map. $\mathbf{w}$ is the direction of steepest ascent. The boundary is a contour line at height 0, and the steepest direction is always perpendicular to contour lines, exactly as on a hiking map.

Signed distance to the boundary

Decompose any point as its projection $\mathbf{x}_\perp$ on the boundary plus a step r along the unit normal:

$$\mathbf{x} = \mathbf{x}_\perp + r \frac{\mathbf{w}}{\|\mathbf{w}\|} \Rightarrow f(\mathbf{x}) = \underbrace{\mathbf{w}^\top \mathbf{x}_\perp + b}_{=0} + r \frac{\mathbf{w}^\top \mathbf{w}}{\|\mathbf{w}\|} = r \|\mathbf{w}\|.$$

Result

$$r(\mathbf{x}) = \frac{f(\mathbf{x})}{\|\mathbf{w}\|} = \frac{\mathbf{w}^\top \mathbf{x} + b}{\|\mathbf{w}\|} \quad (\text{signed geometric distance}).$$

- The raw score $f(\mathbf{x})$ is a distance measured in “units of $\|\mathbf{w}\|$ ”.
- Multiplying y_i in gives $y_i r(\mathbf{x}_i) > 0$ exactly when $\mathbf{x}_i$ is correctly classified.

Worked problem 1: distances by hand

Let $\mathbf{w} = (3, 4)$, $b = -10$, so $\|\mathbf{w}\| = \sqrt{9 + 16} = 5$.

$\mathbf{x}$	y	$f(\mathbf{x}) = 3x_1 + 4x_2 - 10$	distance $f/5$	$yf(\mathbf{x})$	verdict
(2, 2)	+1	$6 + 8 - 10 = 4$	0.8	4	correct, beyond gutter
(1, 1)	-1	$3 + 4 - 10 = -3$	-0.6	3	correct, beyond gutter
(4, 0)	+1	$12 - 10 = 2$	0.4	2	correct
(2, 1)	+1	$6 + 4 - 10 = 0$	0	0	on the boundary

Try it: rescaling

Replace $(\mathbf{w}, b)$ by $(6, 8, -20)$. For $(2, 2)$, $f = 12 + 16 - 20 = 8$ and $\|\mathbf{w}\| = 10$, so the distance is still 0.8. **The line and the geometry are unchanged; only the raw score doubled.**

Scale freedom and the canonical normalisation

- $(\mathbf{w}, b)$ and $(k\mathbf{w}, kb)$ with $k > 0$ describe the *same* line and give the same predictions.
- So the raw score has an arbitrary unit. We fix the unit by a convention:

Canonical scaling

Choose $(\mathbf{w}, b)$ so that the training points *closest* to the boundary satisfy

$$y_i (\mathbf{w}^\top \mathbf{x}_i + b) = 1.$$

Intuition

This is like deciding to measure lengths in “street half-widths”. After this choice, the gutters are exactly the lines $f(\mathbf{x}) = +1$ and $f(\mathbf{x}) = -1$, and every training point satisfies $y_i f(\mathbf{x}_i) \geq 1$.

Where we are

1. The big idea: the widest street
2. Geometry of a linear classifier
3. Maximum margin: the hard-margin SVM
4. Support vectors and the dual view
5. Soft margin: allowing mistakes at a price
6. Kernels: straight lines in a curved world
7. Practice: preprocessing, evaluation and code
8. Comparative study and exercises
9. Wrap-up

How wide is the street?

Take $\mathbf{x}_+$ on the positive gutter ($f = +1$) and $\mathbf{x}_-$ on the negative gutter ($f = -1$).
Subtract:

$$\mathbf{w}^\top (\mathbf{x}_+ - \mathbf{x}_-) = 2.$$

The width is the projection of $\mathbf{x}_+ - \mathbf{x}_-$ on the unit normal:

$$\text{width} = \frac{\mathbf{w}^\top (\mathbf{x}_+ - \mathbf{x}_-)}{\|\mathbf{w}\|} = \frac{2}{\|\mathbf{w}\|}.$$

Key consequence

Wide street $\iff$ small $\|\mathbf{w}\|$.

Each gutter lies at distance $1/\|\mathbf{w}\|$ from the centre line.

From a geometric wish to an optimisation problem

Wish: the widest street such that every house is on its side and not on the street.

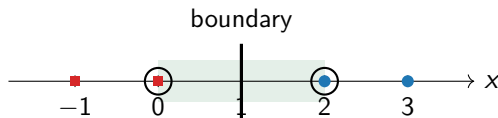
$$\max_{\mathbf{w}, b} \frac{2}{\|\mathbf{w}\|} \iff \boxed{\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{s.t.} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 \quad \forall i}$$

- Maximising $2/\|\mathbf{w}\|$ is the same as minimising $\|\mathbf{w}\|$, and we square it (with $\frac{1}{2}$) to get a smooth **convex quadratic**: one global optimum, no local traps.
- The trick with y_i : the single constraint $y_i f(\mathbf{x}_i) \geq 1$ encodes both

$$f(\mathbf{x}_i) \geq +1 \text{ if } y_i = +1 \quad \text{and} \quad f(\mathbf{x}_i) \leq -1 \text{ if } y_i = -1.$$

Worked problem 2: a one-dimensional SVM by hand

Data on a line: $x = -1, x = 0$ (class -1) and $x = 2, x = 3$ (class $+1$).



1. **Guess the support vectors:** the closest opposite pair, $x = 0$ and $x = 2$.
2. **Make them touch the gutters:** $w \cdot 2 + b = +1$ and $w \cdot 0 + b = -1$.
3. **Solve:** $b = -1$, $w = 1$. Boundary: $x + (-1) = 0 \Rightarrow x = 1$ (the midpoint).
4. **Check the others:** $f(3) = 2 \geq 1$ and $f(-1) = -2 \leq -1$. All constraints hold.
5. **Margin:** $2/|w| = 2$, exactly the gap between 0 and 2.

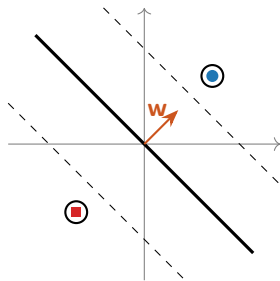
Worked problem 3: two points in the plane

$\mathbf{x}_1 = (1, 1)$, $y_1 = +1$ and $\mathbf{x}_2 = (-1, -1)$, $y_2 = -1$.

1. By symmetry, the best line is the perpendicular bisector: $b = 0$ and $\mathbf{w} = (a, a)$.
2. Constraint for $\mathbf{x}_1$: $a + a = 1 \Rightarrow a = \frac{1}{2}$. So $\mathbf{w} = (\frac{1}{2}, \frac{1}{2})$.
3. $\|\mathbf{w}\| = \frac{1}{\sqrt{2}}$, margin = $2/\|\mathbf{w}\| = 2\sqrt{2} \approx 2.83$.
4. Check: the distance between the two points is $\sqrt{2^2 + 2^2} = 2\sqrt{2}$.

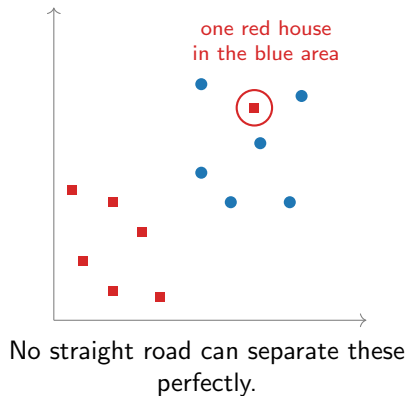
Intuition

With one point per class, the widest street is the whole gap between them, and the boundary sits right in the middle.



When the hard margin breaks

- The hard-margin problem requires *every* point to be outside the street.
- A single mislabelled or unusual point can make this **impossible** (no feasible solution).
- Even when feasible, one outlier can force a very **narrow**, tilted street.
- Real data (overlapping classes, noisy labels) almost never looks like the perfect picture.



Where we are

1. The big idea: the widest street
2. Geometry of a linear classifier
3. Maximum margin: the hard-margin SVM
- 4. Support vectors and the dual view**
5. Soft margin: allowing mistakes at a price
6. Kernels: straight lines in a curved world
7. Practice: preprocessing, evaluation and code
8. Comparative study and exercises
9. Wrap-up

Who holds the street in place?

Introduce one multiplier $\alpha_i \geq 0$ per constraint (Lagrange). The optimality (KKT) conditions give:

$$\mathbf{w} = \sum_i \alpha_i y_i \mathbf{x}_i, \quad \sum_i \alpha_i y_i = 0, \quad \alpha_i (y_i f(\mathbf{x}_i) - 1) = 0.$$

- The last condition (complementary slackness): either $\alpha_i = 0$, or the point sits *exactly* on its gutter.
- So $\mathbf{w}$ is built only from points on the gutters: the **support vectors**.

Physical picture

Think of α_i as the *force* with which house i pushes on the street. Houses far away do not touch it, so they push with force 0. The balance $\sum_i \alpha_i y_i = 0$ says blue pushes and red pushes cancel: the street is in equilibrium, like a tent held by a few pegs.

The dual problem: data enter only through dot products

Substituting $\mathbf{w} = \sum_i \alpha_i y_i \mathbf{x}_i$ back gives the **dual**:

$$\max_{\alpha} \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j \underbrace{\mathbf{x}_i^{\top} \mathbf{x}_j}_{\text{dot product}} \quad \text{s.t.} \quad \alpha_i \geq 0, \quad \sum_i \alpha_i y_i = 0.$$

And predictions become

$$f(\mathbf{x}) = \sum_{i \in S} \alpha_i y_i \mathbf{x}_i^{\top} \mathbf{x} + b, \quad S = \{i : \alpha_i > 0\}.$$

Remember this observation

The training points appear *only* inside dot products $\mathbf{x}_i^{\top} \mathbf{x}_j$. This is the door through which kernels will enter in Section 6.

Worked problem 4: the dual on two points

Back to $\mathbf{x}_1 = (1, 1)$, $y_1 = +1$ and $\mathbf{x}_2 = (-1, -1)$, $y_2 = -1$.

1. Balance: $\alpha_1 y_1 + \alpha_2 y_2 = 0 \Rightarrow \alpha_1 = \alpha_2 = \alpha$.
2. Weights: $\mathbf{w} = \alpha(1, 1) - \alpha(-1, -1) = 2\alpha(1, 1)$.
3. We found $\mathbf{w} = (\frac{1}{2}, \frac{1}{2})$ earlier, so $2\alpha = \frac{1}{2} \Rightarrow \alpha = \frac{1}{4}$.
4. Both $\alpha_i > 0$: both points are support vectors, as expected.

Thought experiment for class

Add a third blue point at $(5, 5)$. Its constraint is $f(5, 5) = 5 \geq 1$: satisfied with room to spare, so its $\alpha = 0$. The solution does not change at all. Now move $(1, 1)$ to $(2, 2)$: the street widens. **Only support vectors change the model.**

Where we are

1. The big idea: the widest street
2. Geometry of a linear classifier
3. Maximum margin: the hard-margin SVM
4. Support vectors and the dual view
5. Soft margin: allowing mistakes at a price
6. Kernels: straight lines in a curved world
7. Practice: preprocessing, evaluation and code
8. Comparative study and exercises
9. Wrap-up

Slack variables: paying fines for sitting on the street

Give each point a slack $\xi_i \geq 0$ measuring how much it violates its gutter:

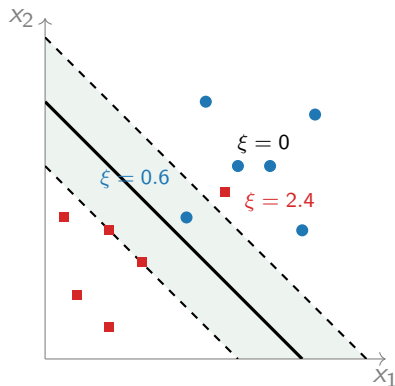
$$\min_{\mathbf{w}, b, \xi} \underbrace{\frac{1}{2} \|\mathbf{w}\|^2}_{\text{narrowness}} + C \underbrace{\sum_i \xi_i}_{\text{total violation}} \quad \text{s.t.} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0.$$

Intuition

The city can still build a wide street, but every house standing on the street, or on the wrong side, pays a **fine** proportional to how deep it intrudes. C is the **price per unit of violation**.

- $\xi_i = 0$: on or beyond its own gutter (no fine).
- $0 < \xi_i < 1$: on the street, but still on the correct side of the centre line.
- $\xi_i > 1$: across the centre line, i.e. **misclassified**.

Reading slacks on a picture



Use $\mathbf{w} = (1, 1)$, $b = -4$, so
 $f(\mathbf{x}) = x_1 + x_2 - 4$ and $\xi = \max(0, 1 - yf)$.

$\mathbf{x}$	y	f	yf	ξ
(3, 3)	+1	2	2	0
(2.2, 2.2)	+1	0.4	0.4	0.6
(2.8, 2.6)	-1	1.4	-1.4	2.4

- Blue (2.2, 2.2): on the street, correct side.
- Red (2.8, 2.6): deep in blue territory, misclassified.

Worked problem 5: what does C change?

Same street: $\frac{1}{2}\|\mathbf{w}\|^2 = \frac{1}{2}(1 + 1) = 1$, and total slack $0 + 0.6 + 2.4 = 3$.

	width cost $\frac{1}{2}\ \mathbf{w}\ ^2$	finest $C \sum \xi_i$	objective
$C = 0.1$	1	0.3	1.3
$C = 1$	1	3	4
$C = 10$	1	30	31

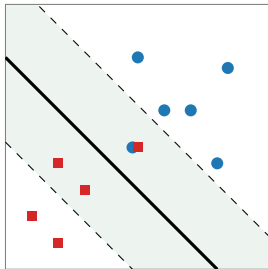
- With $C = 0.1$ fines are cheap: the optimiser happily keeps a wide street and tolerates violators.
- With $C = 10$ fines dominate: the optimiser will narrow and tilt the street to reduce violations, even if that means a larger $\|\mathbf{w}\|$.

Intuition

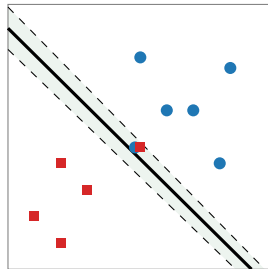
C is a **trust dial**. Small C : “I do not fully trust individual labels, keep it simple.”

Large C : “every training point matters, fit them closely.”

Small C versus large C in pictures



small C : wide street,
a few violations accepted



large C : narrow street,
bent towards the hard points

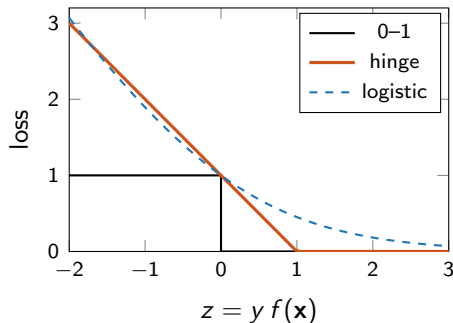
Schematic. Neither is “right”: the best C is chosen by cross-validation.

The hinge loss: the same model seen as “loss + regulariser”

At the optimum each slack equals $\xi_i = \max(0, 1 - y_i f(\mathbf{x}_i))$. Substituting gives an unconstrained problem:

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_i \max(0, 1 - y_i f(\mathbf{x}_i)).$$

- $y_i f \geq 1$: confident and correct, **zero** loss.
- $0 \leq y_i f < 1$: correct but too close, small linear penalty.
- $y_i f < 0$: wrong, penalty grows linearly with how wrong.
- Compare: 0–1 loss (what we care about, but not optimisable) and logistic loss (logistic regression).



Worked problem 6: hinge losses

Three training points have signed scores $y_i f(\mathbf{x}_i) = (1.5, 0.4, -0.2)$.

$y_i f(\mathbf{x}_i)$	hinge $\max(0, 1 - y_i f)$	correct side?	interpretation
1.5	0	yes	safely outside the street
0.4	0.6	yes	on the street, small fine
-0.2	1.2	no	across the centre line

Total hinge loss = 1.8. With $C = 1$ this adds 1.8 to the objective; with $C = 10$ it adds 18.

Caution about outliers

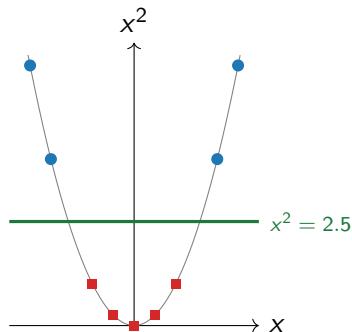
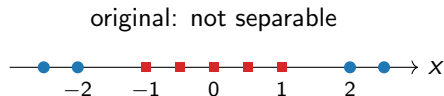
Hinge loss grows only linearly (less than squared loss), but a point with $y_i f = -20$ still costs 21. Inspect extreme points before trusting a large C .

Where we are

1. The big idea: the widest street
2. Geometry of a linear classifier
3. Maximum margin: the hard-margin SVM
4. Support vectors and the dual view
5. Soft margin: allowing mistakes at a price
6. **Kernels: straight lines in a curved world**
7. Practice: preprocessing, evaluation and code
8. Comparative study and exercises
9. Wrap-up

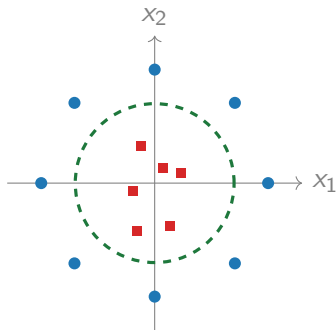
When no straight line works

Example. Red points near 0, blue points far from 0. No single threshold on x separates them.



Idea. Add a new feature $\phi(x) = (x, x^2)$. In the new space a *straight* line $x^2 = 2.5$ separates the classes. Back in the original space it corresponds to the rule $|x| > 1.58$: a *nonlinear* boundary.

Same idea in two dimensions: a ring around a disk



- Classes are a disk and a surrounding ring. Any straight line fails.
- Add $z = x_1^2 + x_2^2$ (squared radius).
- Inner points have $z < 1$, outer points have $z \approx 4$: the flat plane $z = 2$ separates them.
- Seen from above, that plane is the circle $x_1^2 + x_2^2 = 2$.

Intuition

Lift the data into more dimensions, cut with a flat sheet, then look back down: the cut looks curved.

The catch: explicit features can explode

- All polynomial features up to degree d in p variables: $\binom{p+d}{d}$ features.
- Handwritten digits: $p = 64$ pixels, degree 3 gives $\binom{67}{3} = 47,905$ features per image.
- The RBF kernel corresponds to an **infinite**-dimensional feature map.

The escape route

Recall the dual: the data appear *only* through dot products $\mathbf{x}_i^\top \mathbf{x}_j$. If we can compute $\langle \phi(\mathbf{x}_i), \phi(\mathbf{x}_j) \rangle$ directly from $\mathbf{x}_i, \mathbf{x}_j$, we never need to build ϕ .

$$K(\mathbf{x}, \mathbf{x}') = \langle \phi(\mathbf{x}), \phi(\mathbf{x}') \rangle.$$

Worked problem 7: checking the kernel trick numerically

Degree-2 polynomial kernel in $\mathbb{R}^2$: $K(\mathbf{x}, \mathbf{z}) = (\mathbf{x}^\top \mathbf{z})^2$. Its feature map is

$$\phi(\mathbf{x}) = (x_1^2, \sqrt{2} x_1 x_2, x_2^2).$$

Take $\mathbf{x} = (1, 2)$ and $\mathbf{z} = (3, 1)$.

Long way (build features)

$$\phi(\mathbf{x}) = (1, 2\sqrt{2}, 4)$$

$$\phi(\mathbf{z}) = (9, 3\sqrt{2}, 1)$$

$$\langle \phi(\mathbf{x}), \phi(\mathbf{z}) \rangle = 9 + 12 + 4 = 25$$

Short way (kernel)

$$\mathbf{x}^\top \mathbf{z} = 3 + 2 = 5$$

$$K(\mathbf{x}, \mathbf{z}) = 5^2 = 25$$

Intuition

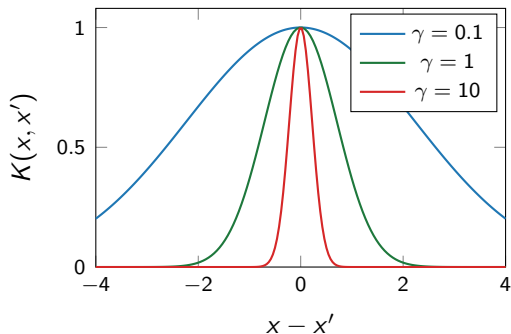
Same number, but the short way never visited the feature space. For degree 3 on digit images the short way is one dot product of length 64 instead of 47,905.

The three kernels you will use

kernel	formula	how to think about it
Linear	$\mathbf{x}^\top \mathbf{x}'$	Plain SVM, a flat boundary. Strong baseline when features are many.
Polynomial	$(\gamma \mathbf{x}^\top \mathbf{x}' + r)^d$	Uses interactions of features up to degree d (e.g. “pixel A <i>and</i> pixel B”).
RBF (Gaussian)	$\exp(-\gamma \ \mathbf{x} - \mathbf{x}'\ ^2)$	“How close are you?”: 1 for identical points, $\rightarrow 0$ far away.

A kernel is best understood as a **similarity measure**. The SVM then says: “a new point gets the label of the support vectors it most resembles, weighted by their importance α_i .”

The RBF kernel and the meaning of γ



- γ controls the **reach** of each training point, roughly $1/\sqrt{\gamma}$.
- **Small** γ : wide bumps, every point influences a large region, boundary is smooth (can underfit).
- **Large** γ : narrow spikes, each point influences only its tiny neighbourhood, boundary wraps around individual points (can overfit).

Worked problem 8: RBF similarity

One-dimensional points $x = 0$ and $x' = 2$, so $\|x - x'\|^2 = 4$.

γ	$K = e^{-4\gamma}$	reading
0.05	$e^{-0.2} \approx 0.819$	“very similar”
0.5	$e^{-2} \approx 0.135$	“somewhat related”
5	$e^{-20} \approx 2 \times 10^{-9}$	“strangers”

Lesson

The points did not move, only γ did. And if we rescale x (e.g. metres to millimetres), $\|x - x'\|^2$ changes by a factor 10^6 . γ **only means something once features are scaled.**

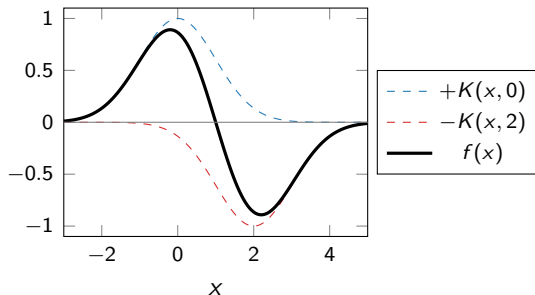
Worked problem 9: an RBF SVM predicts by weighted voting

Two support vectors: $x_1 = 0$ ($y = +1$) and $x_2 = 2$ ($y = -1$), with $\alpha_1 = \alpha_2 = 1$, $b = 0$, $\gamma = 0.5$:

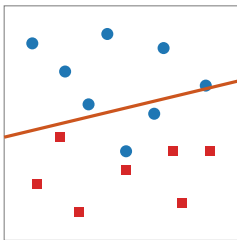
$$f(x) = e^{-0.5x^2} - e^{-0.5(x-2)^2}.$$

x	vote of x_1	vote of x_2	$f(x)$
0.5	0.882	-0.325	+0.558
1	0.607	-0.607	0
1.5	0.325	-0.882	-0.558

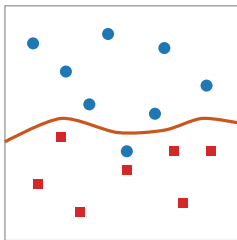
Each support vector “votes” with strength equal to its similarity. The boundary sits where the votes balance.



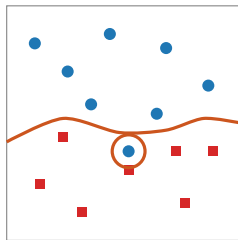
C and γ together: underfit, good fit, overfit



small γ , small C
underfit



moderate γ , C
good



large γ , large C
overfit

- Left: a boundary that is too simple misses the true shape.
- Middle: follows the trend, ignores the isolated blue point (probably noise).
- Right: draws a private bubble around the isolated point: perfect on training data, fragile on new data.
- Because C and γ interact, **tune them jointly** on a grid with cross-validation.

Where we are

1. The big idea: the widest street
2. Geometry of a linear classifier
3. Maximum margin: the hard-margin SVM
4. Support vectors and the dual view
5. Soft margin: allowing mistakes at a price
6. Kernels: straight lines in a curved world
7. Practice: preprocessing, evaluation and code
8. Comparative study and exercises
9. Wrap-up

Why feature scaling is not optional

Two customers described by (age in years, income in euros): $A = (30, 40,000)$ and $B = (60, 41,000)$.

Unscaled

$$\|A - B\|^2 = 30^2 + 1000^2 = 900 + 10^6$$

Income is 99.9% of the distance: a 30-year age gap is invisible to an RBF kernel.

Standardisation: $x_j^{\text{scaled}} = (x_j - \mu_j^{\text{train}}) / s_j^{\text{train}}$.

Standardised (sd age = 15, sd income = 15,000)

$$\|A - B\|^2 = 2^2 + 0.067^2 \approx 4.004$$

Now age dominates, as common sense suggests.

Rule

Compute μ and s on the **training data only**; otherwise test information leaks into the model.

Data leakage: spot the difference

Wrong: scaler sees all data

```
X_s = StandardScaler().fit_transform(X)
X_tr, X_te, y_tr, y_te = train_test_split(
    X_s, y, stratify=y, random_state=42)
cross_val_score(SVC(), X_tr, y_tr, cv=5)
```

Test-set means and variances were used to transform the training data, and every CV fold was scaled using its own validation part.

Right: scaling inside a pipeline

```
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, stratify=y, random_state=42)
pipe = make_pipeline(StandardScaler(), SVC())
cross_val_score(pipe, X_tr, y_tr, cv=5)
```

In each fold, the scaler is refitted on that fold's training part only. The test set is untouched until the very end.

Intuition

The test set is a sealed exam paper. Computing its average before the exam, even “just for scaling”, is peeking.

1. Reserve a **stratified** held-out test set (class proportions preserved).
2. Put scaling and the SVM in **one pipeline**.
3. Search hyperparameters (C , γ , kernel) with **cross-validation on training data only**.
4. Refit the selected pipeline on all training data.
5. Evaluate **once** on the test set; report a baseline and per-class metrics.

Beware

Some data need special splits: patient-level (several scans from one patient), grouped, or chronological (no training on the future).

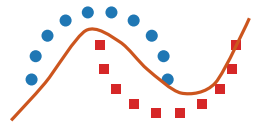
Multiclass problems and what the scores mean

- An SVM is natively binary. SVC uses **one-versus-one**: one classifier for each pair of classes, then a vote.
 - Wine, 3 classes: $3 \cdot 2/2 = 3$ classifiers.
 - Digits, 10 classes: $10 \cdot 9/2 = 45$ classifiers.
- `decision_function` returns signed scores (distances up to scale), **not probabilities**.
- `probability=True` fits an extra calibration step (internal cross-validation): slower, and can disagree slightly with `predict`.
- Accuracy can hide problems when classes are imbalanced: check **per-class recall**, **macro F1** and the **confusion matrix**.

Guided implementation: two moons

- A 2D synthetic dataset: two interleaved half-circles. We can see every boundary.
- Compare a scaled **linear** SVM with a scaled **RBF** SVM.
- Plot decision scores and circle the support vectors.
- Vary C and γ and watch underfitting and overfitting happen.

Companion notebook:
`SVM_Guided_Colab.ipynb`.



RBF boundary (schematic)

Guided code 1: split and fit

```
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

X, y = make_moons(n_samples=400, noise=0.23, random_state=7)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=42)

linear = make_pipeline(StandardScaler(), SVC(kernel="linear", C=1))
rbf = make_pipeline(StandardScaler(), SVC(kernel="rbf", C=1, gamma="scale"))
linear.fit(X_tr, y_tr); rbf.fit(X_tr, y_tr)

print("linear test acc:", linear.score(X_te, y_te))
print("rbf test acc:", rbf.score(X_te, y_te))
```

Predict before running: which one wins on moons, and why?

Guided code 2: look inside the model

```
import matplotlib.pyplot as plt
from sklearn.inspection import DecisionBoundaryDisplay

svc = rbf.named_steps["svc"]
print("support vectors per class:", svc.n_support_)
print("fraction of training points that are SVs:",
      svc.n_support_.sum() / len(X_tr))

disp = DecisionBoundaryDisplay.from_estimator(
    rbf, X, response_method="decision_function",
    plot_method="contour", levels=[-1, 0, 1],
    linestyle=["--", "-", "--"])
disp.ax_.scatter(X_tr[:, 0], X_tr[:, 1], c=y_tr, cmap="coolwarm", s=15)
sv = rbf.named_steps["standardscaler"].inverse_transform(svc.support_vectors_)
disp.ax_.scatter(sv[:, 0], sv[:, 1], facecolors="none", edgecolors="k", s=60)
plt.show()
```

The three contour levels $-1, 0, +1$ are exactly the two gutters and the centre line. Support vectors are stored in scaled units, hence the `inverse_transform`.

Guided code 3: tune C and γ correctly

```
from sklearn.model_selection import GridSearchCV, StratifiedKFold

pipe = make_pipeline(StandardScaler(), SVC(kernel="rbf"))
grid = {"svc__C": [0.1, 1, 10, 100],
        "svc__gamma": [0.01, 0.1, 1, 10]}
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=0)

search = GridSearchCV(pipe, grid, cv=cv, scoring="f1_macro")
search.fit(X_tr, y_tr) # training data only
print(search.best_params_, search.best_score_) # validation score
print("held-out:", search.score(X_te, y_te)) # look ONCE
```

- Grids are usually **logarithmic**: C and γ matter by orders of magnitude.
- Plot `search.cv_results_["mean_test_score"]` as a heatmap over (C, γ) : you will see a diagonal “ridge” of good values, a picture of their interaction.

Choosing a kernel: rules of thumb

Start linear when

- there are many features relative to samples (text, genomics);
- you need interpretable coefficients (after scaling);
- the dataset is large: LinearSVC scales far better.

Try RBF when

- features are few to moderate and relationships look nonlinear;
- the dataset is modest (kernel SVC training grows between n^2 and n^3);
- you can afford a joint (C, γ) search.

Intuition

Always keep the linear SVM (and a trivial baseline) in your results table. If RBF does not clearly beat it, prefer the simpler model.

Where we are

1. The big idea: the widest street
2. Geometry of a linear classifier
3. Maximum margin: the hard-margin SVM
4. Support vectors and the dual view
5. Soft margin: allowing mistakes at a price
6. Kernels: straight lines in a curved world
7. Practice: preprocessing, evaluation and code
- 8. Comparative study and exercises**
9. Wrap-up

Comparative study: three datasets

Dataset	Samples	Features	Task
Breast cancer Wisconsin	569	30	binary
Wine	178	13	three classes
Handwritten digits	1,797	64	ten classes

Use the same split seed, training-only scaling, stratified cross-validation and test metrics for each dataset.

Differences in size and task mean raw accuracy comparisons do not show that one dataset is “easier” in any universal sense.

Source: scikit-learn dataset documentation.

Study questions and measurements

- Which dataset favours linear versus RBF under the chosen search budget?
- How sensitive are the test scores to C and γ ?
- What **fraction** of training points become support vectors? (A high fraction often signals overlap or overfitting.)
- Does a high macro F1 coincide with balanced per-class recall?
- How do training time and the number of support vectors vary?

Discipline

Record validation and held-out scores separately. Never select models by repeatedly looking at the test set.

Exercise 1: wine

Dataset: `load_wine()` (three classes, 13 numeric features).

1. Create a stratified 80/20 split and a majority-class baseline (`DummyClassifier`).
2. Compare scaled linear and RBF SVC pipelines with five-fold cross-validation.
3. Tune $C \in \{0.1, 1, 10\}$ and, for RBF, $\gamma \in \{0.01, 0.1, 1\}$.
4. Report held-out macro F1, accuracy, a confusion matrix and support-vector counts.
5. Explain one class-specific error and whether the selected kernel helped.

Hint. Wine features have very different ranges (proline is in the hundreds). Try once *without* scaling and compare: this is the scaling slide in action.

Exercise 2: handwritten digits

Dataset: `load_digits()` (ten classes, 64 pixel intensities from 0 to 16).

1. Display example 8×8 images and make a stratified 80/20 split.
2. Compare scaled linear SVC and RBF SVC using the same folds.
3. Tune $C \in \{0.1, 1, 10\}$ and RBF $\gamma \in \{0.001, 0.01\}$.
4. Report held-out macro F1 and the most confused pair of digits.
5. Display at least five misclassified images and discuss a failure mode.

Hint. How many binary classifiers does one-versus-one train here? What does a support vector *look like* as an image? (Try plotting a few.)

- The executed exercise notebook.
- A concise results table: baseline, linear SVM, RBF SVM; validation and test scores in separate columns.
- A short interpretation of errors, supported by at least one figure.
- Fixed split seeds; scaling fitted only inside cross-validation.

Guided, study and two student exercise notebooks accompany these slides.

Where we are

1. The big idea: the widest street
2. Geometry of a linear classifier
3. Maximum margin: the hard-margin SVM
4. Support vectors and the dual view
5. Soft margin: allowing mistakes at a price
6. Kernels: straight lines in a curved world
7. Practice: preprocessing, evaluation and code
8. Comparative study and exercises

9. Wrap-up

Check your understanding

1. If you double both $\mathbf{w}$ and b , what happens to the boundary, to $f(\mathbf{x})$ and to the geometric distance?
2. A training point has $y_i f(\mathbf{x}_i) = 3$. What is its hinge loss and its α_i ?
3. You delete a non-support vector from the training set and retrain. What changes?
4. Your RBF SVM gets 100% training accuracy and uses 95% of points as support vectors. Diagnosis and remedy?
5. Why is it wrong to standardise the full dataset before cross-validation?
6. For $\mathbf{x} = (2, 0)$, $\mathbf{z} = (1, 3)$, compute $(\mathbf{x}^\top \mathbf{z})^2$ both directly and through ϕ .

1. Boundary unchanged; $f(\mathbf{x})$ doubles; geometric distance unchanged (numerator and $\|\mathbf{w}\|$ both double).
2. Hinge = $\max(0, 1 - 3) = 0$; the point is outside the street, so $\alpha_i = 0$.
3. Nothing: the solution depends only on support vectors.
4. Overfitting: each point has its own bubble. Decrease γ and/or C , choose them by cross-validation.
5. The scaler would learn means and variances from validation (and test) points: information leaks, and scores become optimistic.
6. $\mathbf{x}^\top \mathbf{z} = 2$, so $K = 4$. Via ϕ : $(4, 0, 0) \cdot (1, 3\sqrt{2}, 9) = 4$. Same.

One-slide summary

Concept	One-sentence meaning
Margin $2/\ \mathbf{w}\ $	Width of the empty street; SVM makes it as large as possible.
Support vectors	Points on or inside the gutters; the only ones that shape the model.
Slack ξ_i , hinge	How far a point intrudes into the street; zero when safely outside.
C	Price per violation: small C tolerant and smooth, large C strict and wiggly.
Kernel K	A similarity that acts as a dot product in a richer feature space.
γ (RBF)	Reach of each point's influence: small is smooth, large is local.
Pipeline + CV	Scale and tune inside folds; open the test set once.

- scikit-learn user guide, Support Vector Machines:
<https://scikit-learn.org/stable/modules/svm.html>
- sklearn.svm.SVC API:
<https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVC.html>
- Toy datasets:
https://scikit-learn.org/stable/modules/datasets/toy_dataset.html
- C. Cortes and V. Vapnik, "Support-vector networks", *Machine Learning* 20, 1995.
- T. Hastie, R. Tibshirani, J. Friedman, *The Elements of Statistical Learning*, 2nd ed., Ch. 12.
- B. Schölkopf and A. Smola, *Learning with Kernels*, MIT Press, 2002.